

Universitat Politècnica de València
ETSI Aeroespacial y Diseño Industrial

Sistema de Terminación de Vuelo

Flight Termination System



Aportación al ejercicio personal

La aportación principal de Raúl Martínez Martínez fue la programación del microcontrolador. El proyecto se realizó conjuntamente con Miguel Vela García y Julen Lorenzo Moa.

Asignatura: Ingeniería Electrónica
Curso: 2.º Ingeniería Aeroespacial
Autores: Raúl Martínez Martínez, Miguel Vela García y Julen Lorenzo Moa
Fecha: 10 de octubre de 2026

Índice

1. Introducción	3
2. Descripción teórica del sistema FTS	3
3. Simplificación aplicada para la simulación en Proteus	3
4. Diagrama de bloques del hardware	4
5. Circuito electrónico en Proteus	5
6. Diagrama de flujo del funcionamiento	6
7. Descripción de los elementos hardware	8
7.1. Microcontrolador ATmega328P	8
7.2. Sensor PT100	8
7.3. Puente de Wheatstone	8
7.4. Amplificador operacional TL082	8
7.5. Interruptores ARM y FIRE	9
7.6. Puertas lógicas	9
7.7. Optoacopladores PC817	9
7.8. MOSFET y motor DC	9
7.9. Diodo de protección 1N4007	9
7.10. Servomotor	10
7.11. Registros de desplazamiento 4094	10
7.12. Displays de 7 segmentos	10
7.13. ULN2003A	10
7.14. LEDs de estado	10
8. Descripción del código del microcontrolador	11
8.1. Configuración inicial	11
8.2. Definición de umbrales y mapa de pines	11
8.3. Configuración de puertos, ADC y temporizadores	12
8.4. Bucle principal	12
8.5. Interrupción del Timer2	13
8.6. Lectura de temperatura	13
8.7. Control térmico e histéresis	14
8.8. Control proporcional de la compuerta	14
8.9. Estados SAFE, ARMED y FIRE	14
8.10. Visualización mediante registros 4094	14
8.11. Display multiplexado	14
8.12. Resumen del funcionamiento del programa	15
9. Simulación y comprobación del funcionamiento en Proteus	15
9.1. Lectura de temperatura	15
9.2. Control proporcional de la compuerta	16
9.3. Fallo térmico	16
9.4. Estados SAFE, ARMED y FIRE	16
9.5. Limitaciones de la simulación	16

10.Conclusiones	17
11.Bibliografía	17
A. Código completo del microcontrolador	19

1. Introducción

La seguridad operacional constituye uno de los pilares fundamentales en el diseño y desarrollo de sistemas aeroespaciales. En este contexto, el *Flight Termination System* o Sistema de Terminación de Vuelo, conocido como FTS, es un sistema de seguridad cuya función es finalizar el vuelo de un vehículo cuando este se desvía de su trayectoria prevista o supone un riesgo para personas, instalaciones o el entorno.

En un sistema real, el FTS se utiliza principalmente en lanzadores espaciales, cohetes experimentales, misiles y aeronaves no tripuladas. Su activación debe estar muy controlada, ya que un disparo accidental supondría la pérdida de la misión, mientras que un fallo de activación en una situación crítica podría provocar daños graves. Por ello, estos sistemas suelen incorporar redundancia, aislamiento eléctrico, lógica de confirmación y mecanismos de seguridad.

El objetivo de este trabajo es diseñar y simular en Proteus una versión simplificada de un FTS mediante un microcontrolador ATmega328P. La simulación no pretende reproducir todos los detalles de un sistema real, sino implementar sus ideas principales desde el punto de vista de la Ingeniería Electrónica que se ha aprendido durante el curso.

2. Descripción teórica del sistema FTS

Un FTS real está formado por un conjunto de sensores, receptores, unidades de decisión, sistemas de alimentación y actuadores pirotécnicos. Su misión es detectar o recibir una orden de terminación de vuelo y ejecutar una acción irreversible sobre el vehículo.

Desde el punto de vista electrónico, un FTS debe cumplir tres condiciones principales. En primer lugar, debe ser fiable, ya que tiene que responder cuando se produce una situación de emergencia. En segundo lugar, debe ser seguro frente a activaciones accidentales. En tercer lugar, debe estar protegido frente a interferencias, fallos eléctricos y perturbaciones generadas por otros sistemas del vehículo.

Para conseguirlo, se utilizan medidas como señales redundantes, doble confirmación de armado y disparo, aislamiento galvánico, alimentación separada entre control y potencia, protección frente a cargas inductivas y monitorización de variables críticas.

3. Simplificación aplicada para la simulación en Proteus

El sistema real se ha simplificado para poder implementarlo con los conocimientos adquiridos durante el curso. La primera simplificación es que la carga explosiva real se sustituye por un motor DC. Este motor representa visualmente la activación de la carga de terminación, pero no implica ningún proceso destructivo.

La segunda simplificación es la comunicación entre el centro de mando y el vehículo. En un FTS real, la orden podría recibirse inalámbricamente. En este proyecto, la orden se introduce mediante los interruptores físicos ARM y FIRE.

La tercera simplificación afecta al entorno térmico. En un vehículo real habría varios sensores distribuidos en zonas críticas, midiendo la temperatura de la electrónica, la estructura, las baterías o el sistema propulsivo. En nuestro caso se utiliza un único sensor PT100 simulado, acondicionado mediante un puente de Wheatstone y un amplificador operacional.

Además, en un sistema real la apertura de la compuerta de ventilación tendría un efecto físico sobre la temperatura del sistema. Es decir, al abrirse la compuerta aumentaría el flujo de aire, se produciría refrigeración y la temperatura medida por el sensor debería disminuir progresivamente. Ese comportamiento correspondería a un sistema en bucle cerrado, ya que la temperatura controla la apertura del servo y, a su vez, la apertura del servo modifica la temperatura.

En nuestra simulación no se ha modelado ese efecto térmico de realimentación. La temperatura se introduce manualmente mediante el sensor PT100 simulado, y el servomotor actúa únicamente como consecuencia de esa temperatura. Por tanto, el modelo implementado representa la relación directa entre temperatura medida y apertura de la compuerta, pero no contempla la transferencia de calor, la dinámica del flujo de aire ni la evolución temporal de la temperatura tras abrir la compuerta. Esta simplificación se adopta porque modelar correctamente ese comportamiento térmico queda fuera del alcance del trabajo y de los conocimientos necesarios para esta práctica.

La cuarta simplificación corresponde al actuador térmico. El servomotor no actúa sobre un modelo térmico real, sino que representa visualmente la apertura de una compuerta de ventilación. Su ángulo depende de la temperatura medida, cumpliendo así el requisito de que el actuador proporcional dependa de una entrada analógica.

Otra simplificación es la gestión de los tiempos. El programa limita la espera y la activación del motor mediante contadores de iteraciones y retardos. Cada iteración también realiza cálculos, conversiones y tareas de visualización, por lo que las duraciones indicadas en los comentarios del código no equivalen a tiempos medidos. Al finalizar la secuencia se desactivan las salidas y se vuelve a SAFE. La comprobación de las duraciones reales queda pendiente de una medida temporal en Proteus.

4. Diagrama de bloques del hardware

El sistema puede dividirse en varios bloques funcionales. El primer bloque es la etapa de entrada, formada por los interruptores ARM/FIRE y por el sensor de temperatura PT100. El segundo bloque es el acondicionamiento de señal, formado por el puente de Wheatstone y el amplificador operacional TL082. El tercer bloque es el microcontrolador ATmega328P, que actúa como unidad central de procesamiento.

Finalmente, el sistema cuenta con una etapa de visualización, una etapa de actuación térmica mediante servomotor y una etapa de potencia que activa la carga simulada mediante optoacopladores, puertas lógicas, MOSFET y motor DC.

Bloque	Función dentro del sistema
Entrada de mando	Recoge las órdenes ARM y FIRE mediante interruptores redundantes.
Entrada analógica	Mide la temperatura mediante una PT100 simulada.
Acondicionamiento de señal	Adapta la pequeña señal de la PT100 al rango de lectura del ADC del microcontrolador.
Control principal	El ATmega328P procesa entradas, estados, displays y actuadores.
Visualización	Los displays de 7 segmentos y los LEDs muestran temperatura, apertura de compuerta y estado del sistema.
Actuación térmica	El servomotor abre la compuerta de ventilación de forma proporcional a la temperatura.
Etapas de potencia	Los optoacopladores, la lógica digital y el MOSFET activan el motor que simula la carga del FTS.

Tabla 1: Diagrama de bloques funcional del hardware.

5. Circuito electrónico en Proteus

En la Figura 1 se muestra la captura final del circuito completo implementado en Proteus. En este circuito se integran el microcontrolador ATmega328P, el acondicionamiento de señal del sensor PT100, las entradas de ARM y FIRE, la lógica de redundancia, los optoacopladores, la etapa de potencia con MOSFET, el motor DC que simula la carga de terminación, el servomotor de la compuerta de ventilación y los displays de 7 segmentos.

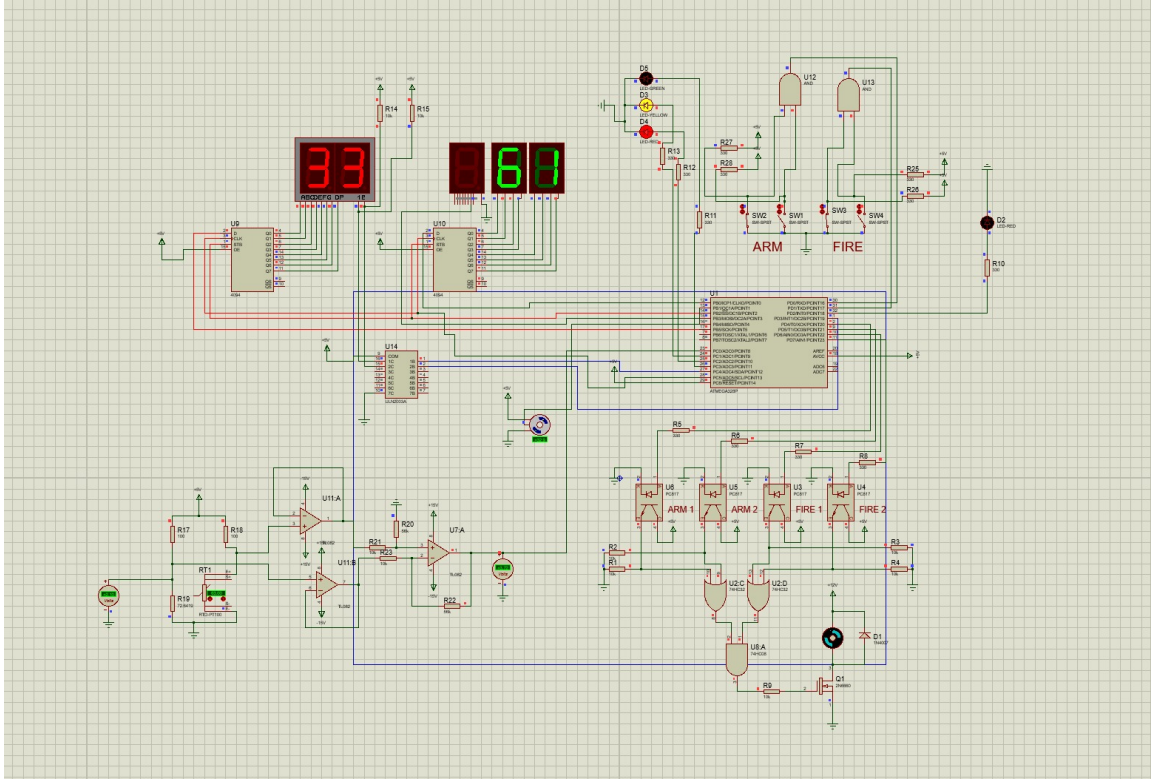


Figura 1: Circuito electrónico completo implementado en Proteus.

6. Diagrama de flujo del funcionamiento

El programa configura los puertos, el ADC y los temporizadores, y comienza con las salidas desactivadas. A partir de la lectura de la PT100 calcula la temperatura y la apertura del servo. También actualiza los displays y el indicador de fallo térmico. La condición térmica incorpora la histéresis descrita en el apartado 8.7.

La lógica de mando utiliza los estados SAFE, ARMED y FIRE y la variable `bloqueo_secuencia`. La figura 2 resume la coordinación entre la actualización de medidas, la consulta de las entradas y la actuación sobre el motor de demostración. El código detallado se incluye en el anexo.

La temperatura se actualiza periódicamente en el bucle principal y vuelve a leerse dentro de los bucles de la secuencia. Estas comprobaciones se ejecutan junto con el cálculo del servo y el refresco de los displays; no constituyen una supervisión continua ni una respuesta instantánea.

La espera y la activación se limitan mediante iteraciones. Su duración incluye el trabajo ejecutado dentro de cada bucle y no se ha medido en esta revisión. Por ese motivo, el diagrama no asigna una duración en segundos. Tampoco incluye una medida de batería, ya que esa adquisición no está implementada en el programa.

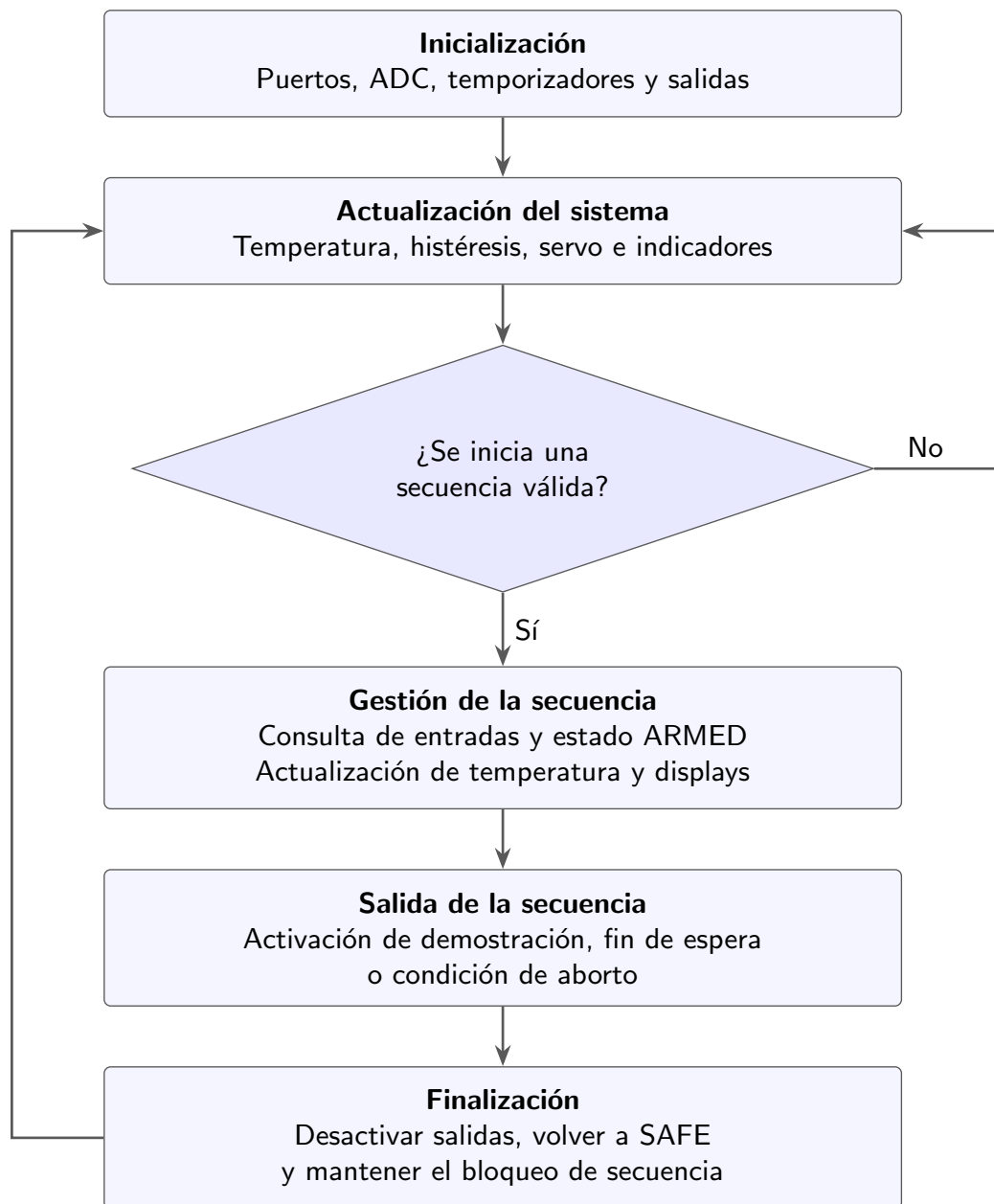


Figura 2: Diagrama funcional resumido de la simulación.

La figura resume el comportamiento del código del anexo. La duración de los bucles no se ha verificado mediante registros temporales.

7. Descripción de los elementos hardware

En este apartado se describen los principales componentes empleados en el circuito electrónico. Cada uno de ellos cumple una función concreta dentro del sistema, ya sea como elemento de entrada, acondicionamiento, control, visualización, aislamiento o actuación.

7.1. Microcontrolador ATmega328P

El ATmega328P actúa como controlador principal del sistema. Se encarga de leer la señal analógica procedente del acondicionamiento del sensor PT100, procesar las señales digitales de ARM y FIRE, activar los LEDs de estado, controlar el servomotor, refrescar los displays y generar las salidas hacia la etapa de optoacopladores.

Este microcontrolador resulta adecuado para la simulación porque dispone de conversor analógico-digital, temporizadores internos y puertos digitales suficientes para implementar la lógica del sistema. En el proyecto se programa mediante registros, configurando directamente los puertos, el ADC y los temporizadores.

7.2. Sensor PT100

La PT100 es un sensor resistivo de temperatura. Su resistencia nominal es de aproximadamente $100, \Omega$ a $0, ^\circ \text{C}$ y aumenta al incrementarse la temperatura. En el sistema simulado se utiliza como entrada analógica principal para representar la temperatura del sistema electrónico.

Como la variación de resistencia de la PT100 es pequeña, no se conecta directamente al ADC del microcontrolador. En su lugar, se utiliza una etapa de acondicionamiento formada por un puente de Wheatstone y un amplificador operacional.

7.3. Puente de Wheatstone

El puente de Wheatstone permite transformar la variación de resistencia de la PT100 en una variación de tensión. Su utilidad principal es comparar la rama donde se encuentra la PT100 con una rama de referencia. De esta forma, se obtiene una señal diferencial asociada al cambio de temperatura.

Este montaje permite reducir la componente continua de la señal y quedarse con una variación más útil para su posterior amplificación. En la práctica, el puente facilita que el microcontrolador pueda interpretar los cambios de temperatura de forma más precisa.

7.4. Amplificador operacional TL082

La señal procedente del puente de Wheatstone es demasiado pequeña para ser leída directamente con buena resolución por el ADC del ATmega328P. Por ello, se utiliza un amplificador operacional TL082 para aumentar la amplitud de la señal y adaptarla al rango de trabajo del microcontrolador.

El objetivo de esta etapa es que la tensión entregada al ADC se mantenga dentro del rango aproximado de 0 a 5, V, aprovechando mejor la resolución de conversión del microcontrolador.

7.5. Interruptores ARM y FIRE

El sistema cuenta con interruptores para las órdenes de ARM y FIRE. La orden ARM representa el armado del sistema, mientras que FIRE representa la autorización final de activación.

Para evitar activaciones accidentales, el sistema exige una secuencia lógica: primero debe activarse ARM y después FIRE. Si no se cumple esta condición, el sistema permanece en estado SAFE y no activa la carga simulada.

7.6. Puertas lógicas

Las puertas lógicas se utilizan para combinar las señales redundantes del sistema. Las puertas OR permiten agrupar señales equivalentes, de forma que si una de ellas está activa, se considera válida la orden correspondiente. La puerta AND final exige que estén presentes simultáneamente las señales de ARM y FIRE antes de activar la etapa de potencia.

Esta estructura permite representar una lógica de seguridad básica, en la que no basta con una única señal aislada para activar el sistema de terminación.

7.7. Optoacopladores PC817

Los optoacopladores PC817 se emplean para aislar eléctricamente el microcontrolador de la etapa de potencia. Internamente, transmiten la señal mediante luz, por lo que no existe una conexión eléctrica directa entre la parte de control y la parte de salida.

Este aislamiento protege al ATmega328P frente a posibles perturbaciones, picos de tensión o interferencias generadas en la zona de potencia. En el circuito se utilizan optoacopladores en las líneas de ARM y FIRE para representar una arquitectura más segura y robusta.

7.8. MOSFET y motor DC

El MOSFET actúa como interruptor electrónico de potencia. Cuando la lógica final confirma que existen señales válidas de ARM y FIRE, el MOSFET conduce y permite el paso de corriente hacia el motor DC.

El motor DC sustituye a la carga explosiva real de un FTS. En la simulación, su giro representa visualmente que la orden de terminación de vuelo se ha ejecutado. Esta sustitución es necesaria por seguridad y por las limitaciones propias de una simulación educativa.

7.9. Diodo de protección 1N4007

El motor DC es una carga inductiva. Cuando se desconecta, puede generar una sobretensión inversa debido al efecto de autoinducción. Para proteger el MOSFET frente a ese pico de tensión, se coloca un diodo 1N4007 en antiparalelo con el motor.

Este diodo proporciona un camino de descarga para la corriente inducida y evita que la sobretensión dañe la etapa de potencia.

7.10. Servomotor

El servomotor representa la compuerta de ventilación del sistema. Su posición depende de la temperatura medida por la PT100. Para temperaturas bajas, la compuerta permanece cerrada. A partir de una temperatura determinada, la apertura aumenta de forma proporcional.

De esta forma se cumple el requisito de que el actuador esté controlado proporcionalmente por una entrada analógica.

7.11. Registros de desplazamiento 4094

Los registros de desplazamiento 4094 permiten controlar displays de 7 segmentos utilizando menos pines del microcontrolador. El ATmega328P envía los datos en serie y el 4094 los convierte en salidas paralelas para activar los segmentos correspondientes.

En el sistema se utilizan para visualizar la temperatura y el porcentaje de apertura de la compuerta. Esta solución permite reducir el número de conexiones necesarias y simplificar el control desde el microcontrolador.

7.12. Displays de 7 segmentos

Los displays de 7 segmentos se utilizan como herramienta de visualización simple. Uno de los bloques muestra la temperatura medida, incluyendo el signo negativo cuando corresponde. El display multiplexado muestra el porcentaje de apertura de la compuerta de ventilación, entre 0 y 99.

El multiplexado consiste en activar alternativamente cada dígito a gran velocidad, de forma que visualmente parece que ambos están encendidos al mismo tiempo, aunque realmente se refrescan de manera secuencial.

7.13. ULN2003A

El ULN2003A se utiliza como etapa de conmutación para seleccionar los dígitos del display multiplexado. Sus salidas son de colector abierto, por lo que funcionan como interruptores a masa.

Cuando una entrada del ULN2003A recibe un nivel alto desde el microcontrolador, la salida correspondiente conecta el común del display a GND y el dígito se enciende. Cuando la entrada está a nivel bajo, la salida queda abierta y el dígito se apaga mediante la resistencia de pull-up conectada a +5V.

7.14. LEDs de estado

El sistema incorpora LEDs para indicar los estados principales. El LED SAFE indica que el sistema está inactivo y seguro. El LED ARMED indica que el sistema está armado, pero aún no ha ejecutado FIRE. El LED FIRE indica que se ha activado la orden de disparo.

Además, se incluye un LED de fallo térmico. Este LED se enciende cuando la temperatura supera el límite de seguridad definido en el programa, avisando de que el sistema se encuentra fuera del rango operativo.

8. Descripción del código del microcontrolador

El código del microcontrolador se ha estructurado por bloques para facilitar su comprensión. En esta sección no se incluye el programa completo, ya que este se incorpora al final del documento como anexo. Aquí se explica la función de cada parte principal del código y su relación con el circuito implementado en Proteus.

8.1. Configuración inicial

En primer lugar, se define la frecuencia de trabajo del microcontrolador:

```
1 #ifndef F_CPU
2 #define F_CPU 16000000UL
3 #endif
```

Esta frecuencia es necesaria para que las funciones de retardo y los temporizadores trabajen correctamente. Posteriormente se incluyen las librerías necesarias para trabajar con registros AVR, interrupciones, retardos, operaciones matemáticas y funciones auxiliares.

Después se declaran las variables globales del sistema. Las variables *VTemp*, *Vout*, *Vref*, *Vpt*, *RxT* y *Treal* se utilizan para transformar la lectura analógica del sensor PT100 en una temperatura real. La variable *T* almacena la temperatura en forma entera para poder mostrarla en los displays.

También se declaran variables de estado:

- *armready*: indica que el sistema está armado.
- *fireready*: indica que se ha autorizado la señal de disparo.
- *tempready*: indica que la temperatura está dentro del rango operativo.

8.2. Definición de umbrales y mapa de pines

A continuación se definen las constantes asociadas al sensor PT100 y a los límites de temperatura. La compuerta comienza a abrirse a partir de 55 °C y alcanza su apertura máxima a 75 °C. Este último valor también se utiliza como límite de seguridad térmica.

```
1 #define TEMP_INICIO_REFRIGERACION 55.0
2 #define TEMP_LIMITE_SERVO 75.0
3 #define TEMP_LIMITE_SEGURIDAD 75.0
4 #define TEMP_MINIMA_SEGURIDAD -40.0
```

El mapa de pines se define mediante directivas *#define*. Esto permite que el código sea más claro, ya que en lugar de trabajar directamente con nombres como *PB0*, *PD4* o *PC5*, se usan nombres relacionados con la función real de cada pin.

Por ejemplo, en el puerto D se encuentran las entradas *ARM* y *FIRE*, el LED de fallo térmico y las salidas hacia los optoacopladores:

```
1 #define ARM_IN PD0
2 #define FIRE_IN PD1
3 #define LED_FALLO_TERM PD2
4 #define DIG2 PD3
5 #define OPTO_ARM1 PD4
6 #define OPTO_ARM2 PD5
```

```

7 #define OPTO_FIRE1          PD6
8 #define OPTO_FIRE2          PD7

```

8.3. Configuración de puertos, ADC y temporizadores

La función `setup()` configura todos los periféricos principales del microcontrolador. Los registros `DDRB`, `DDRC` y `DDRD` determinan qué pines se comportan como entradas y cuáles como salidas.

Las entradas `ARM` y `FIRE` se configuran con resistencias internas de pull-up. Por este motivo, estas entradas son activas en nivel bajo: cuando el interruptor se acciona, el microcontrolador lee un 0 lógico.

```

1 PORTD = (1 << ARM_IN) | (1 << FIRE_IN);

```

El ADC se configura con referencia `AVCC` y prescaler 128:

```

1 ADCSRA = 0b10000111;
2 ADMUX   = 0b01000000;

```

La referencia `AVCC` permite trabajar en el rango de 0 a 5 V. El prescaler 128 se utiliza para reducir la frecuencia del ADC y conseguir una lectura estable de la señal analógica.

El `Timer1` se emplea para generar la señal PWM del servomotor. Se configura con un periodo aproximado de 20 ms, adecuado para el control de servos:

```

1 TCCR1A = (1 << COM1A1);
2 TCCR1B = (1 << WGM13) | (1 << CS11);
3 ICR1    = 20000;

```

El `Timer2` se utiliza como temporizador auxiliar para activar una bandera de actualización aproximadamente cada 200 ms. Esta bandera permite actualizar periódicamente la lectura de temperatura sin bloquear continuamente el programa principal.

8.4. Bucle principal

El bucle principal `loop()` gestiona la secuencia de funcionamiento del sistema. En primer lugar, se actualiza periódicamente el sistema si la bandera `actualizar200ms` está activa. Además, en cada iteración se llama a la función de visualización para refrescar los displays.

La secuencia comienza cuando el operador activa la señal `ARM`. Si la temperatura está dentro del rango operativo, el sistema pasa al estado `ARMED`. En ese momento se activan las dos salidas `ARM` hacia los optoacopladores y se enciende el LED correspondiente.

Durante el estado `ARMED`, el programa comprueba de forma repetida el estado de la temperatura, de la entrada `ARM` y de la entrada `FIRE`. Si `ARM` se desactiva o la temperatura deja de estar dentro del rango permitido, la secuencia se aborta y el sistema vuelve a `SAFE`. Si `FIRE` se activa correctamente mientras el sistema está armado, se activan las dos salidas `FIRE` durante un intervalo breve.

Después de la activación `FIRE`, o si se produce un aborto por temperatura o por desactivación de `ARM`, el sistema desactiva las salidas `ARM` y `FIRE` y vuelve al estado `SAFE`. De esta forma, el microcontrolador mantiene una secuencia básica de seguridad: primero `ARM`, después `FIRE`, y siempre condicionada a que la temperatura sea válida.

```

1 if ((PIND & (1 << ARM_IN)) != 0) {
2     bloqueo_secuencia = 0;
3 }

```

La secuencia de lanzamiento solo se inicia si ARM está activo y no existe bloqueo. Además, el código comprueba que FIRE no estuviera ya activado antes de ARM. Si FIRE está activado de forma prematura, la secuencia se bloquea y se aborta esa vuelta del programa.

Si la temperatura es correcta, el sistema pasa a estado ARMED. En ese momento se activan las dos salidas ARM hacia los optoacopladores y se enciende el LED correspondiente. A partir de ahí, se abre una ventana temporal durante la cual se comprueban tres condiciones: que ARM siga activo, que la temperatura siga siendo válida y que FIRE se pulse correctamente.

Si durante la ventana se baja ARM o aparece un fallo térmico, el sistema aborta la secuencia y vuelve a SAFE. Si FIRE se activa correctamente, se activan las dos salidas FIRE durante un tiempo limitado y después se desactivan.

8.5. Interrupción del Timer2

La interrupción del Timer2 no realiza directamente todos los cálculos del sistema. Su función es contar desbordamientos y activar una bandera cuando se alcanza el tiempo deseado.

```

1 if (contador200ms >= 12) {
2     actualizar200ms = 1;
3     contador200ms = 0;
4 }

```

De esta forma, la interrupción se mantiene corta y el trabajo principal se realiza en el bucle principal. Esto evita que una interrupción demasiado larga interfiera con el refresco de displays o con la lógica ARM/FIRE.

8.6. Lectura de temperatura

La función `leer_temperatura()` se encarga de leer el canal ADC0 y transformar el valor digital obtenido en una temperatura. Primero se inicia la conversión analógica-digital y se espera a que termine:

```

1 ADCSRA |= (1 << ADSC);
2 while ((ADCSRA & (1 << ADSC)) != 0);

```

Después, el valor del ADC se convierte a tensión:

$$V_{out} = 5 \cdot \frac{V_{Temp}}{1023}$$

A partir de esta tensión se calcula la tensión equivalente en el sensor, la resistencia de la PT100 y finalmente la temperatura:

$$T = \frac{R_{xT} - R_0}{R_0 \cdot \alpha}$$

El resultado se guarda en `Treal`, mientras que `T` almacena el valor entero que se muestra en los displays.

8.7. Control térmico e histéresis

La función `actualizar_tempready()` comprueba si la temperatura se encuentra dentro del rango permitido. Si la temperatura supera el límite de seguridad o baja por debajo del mínimo definido, `tempready` pasa a valer 0, se bloquea FIRE y se desactivan las salidas de disparo.

El código incluye histéresis de 2 °C. Esto significa que, cuando el sistema entra en fallo térmico, no vuelve al estado operativo justo al cruzar otra vez el límite, sino que necesita recuperar un pequeño margen de seguridad. Esta medida evita oscilaciones rápidas cuando la temperatura está cerca del umbral.

8.8. Control proporcional de la compuerta

La función `calcular_apertura_y_servo()` calcula la apertura de la compuerta de ventilación. Para temperaturas iguales o menores que 55 °C, la apertura es 0. Entre 55 °C y 75 °C, la apertura aumenta linealmente. Para temperaturas iguales o superiores a 75 °C, la apertura se satura al máximo.

$$\text{Apertura} = \begin{cases} 0, & T \leq 55 \\ \frac{T - 55}{75 - 55} \cdot 99, & 55 < T < 75 \\ 99, & T \geq 75 \end{cases}$$

El valor `apertura` se muestra en el display multiplexado, mientras que `servoCmd` se utiliza para mover el servomotor. Posteriormente, la función `pulso_servo()` actualiza el registro OCR1A, modificando el ancho de pulso PWM aplicado al servo.

8.9. Estados SAFE, ARMED y FIRE

Las funciones `estado_safe()`, `estado_armed()` y `estado_fire()` controlan los LEDs de estado del sistema. En SAFE solo permanece encendido el LED verde de seguridad. En ARMED se enciende el LED de armado. En FIRE se encienden los LEDs que indican que el sistema ha ejecutado la orden de disparo.

Las funciones `activar_arm()`, `desactivar_arm()`, `activar_fire()` y `desactivar_fire()` controlan las salidas hacia los optoacopladores. El sistema utiliza dos salidas ARM y dos salidas FIRE para representar una arquitectura redundante.

8.10. Visualización mediante registros 4094

La función `enviar_4094_doble()` envía información simultáneamente a los registros 4094 asociados al display de temperatura y al display de apertura. Se utilizan dos líneas de datos independientes, una para cada grupo de displays, pero comparten el reloj y el strobe.

Cada bit se introduce mediante un flanco de reloj. Al finalizar el envío de los 8 bits, se activa el strobe para transferir los datos a las salidas del registro.

8.11. Display multiplexado

El porcentaje de apertura se muestra en un display multiplexado de dos dígitos. Para ello, la función `display_apertura()` calcula decenas y unidades, envía el patrón corres-

pondiente al registro 4094 y activa alternativamente cada dígito mediante el ULN2003A.

El multiplexado consiste en encender durante unos milisegundos un dígito y después el otro. Aunque eléctricamente no están encendidos al mismo tiempo, la conmutación es suficientemente rápida para que visualmente parezcan permanecer encendidos de forma continua.

La misma función también controla el signo de la temperatura. Si la temperatura es negativa, se activa el pin asociado al signo; si es positiva, se mantiene apagado.

8.12. Resumen del funcionamiento del programa

En conjunto, el código sigue la siguiente lógica:

1. Leer periódicamente la temperatura.
2. Calcular la apertura proporcional de la compuerta.
3. Actualizar el servo y los displays.
4. Comprobar si la temperatura está dentro del rango operativo.
5. Permitir ARM solo si no hay fallo térmico.
6. Permitir FIRE solo si ARM está activo, FIRE se pulsa en el momento correcto y la temperatura sigue siendo válida.
7. Activar la carga simulada durante un intervalo limitado.
8. Volver a SAFE tras la activación FIRE o tras un aborto por temperatura.

9. Simulación y comprobación del funcionamiento en Proteus

Una vez implementado el circuito y cargado el código en el microcontrolador ATmega328P, se realizó la simulación completa en Proteus. El objetivo principal de esta simulación fue comprobar que los distintos bloques del sistema funcionaban de forma coordinada: lectura de temperatura, visualización, control de la compuerta, lógica ARM/FIRE, bloqueo por temperatura y activación de la carga simulada.

9.1. Lectura de temperatura

La temperatura del sistema se introduce manualmente mediante el sensor PT100 simulado. Al modificar su valor, cambia la tensión de salida del circuito de acondicionamiento. Esta tensión llega a la entrada analógica del microcontrolador, donde se convierte mediante el ADC.

Durante la simulación se comprobó que el valor de temperatura mostrado en los displays variaba correctamente al modificar la PT100. También se verificó que el signo negativo funcionaba cuando la temperatura calculada era inferior a cero.

9.2. Control proporcional de la compuerta

El servomotor representa la apertura de una compuerta de ventilación. En la simulación se comprobó que, para temperaturas bajas, la compuerta permanecía cerrada. A partir de 55,° C, el valor de apertura empezaba a aumentar de forma proporcional.

La apertura se muestra en el display multiplexado como un porcentaje entre 0 y 99. Cuando la temperatura se aproxima al límite superior definido, el display llega al valor máximo y el servomotor alcanza su posición de apertura máxima.

9.3. Fallo térmico

El sistema incorpora una condición de seguridad térmica. Si la temperatura se encuentra fuera del rango operativo, la variable interna `tempready` pasa a valer 0. En ese caso, se enciende el LED de fallo térmico y se bloquea la posibilidad de activar FIRE.

En la simulación se comprobó que, al superar el límite de temperatura, el LED de fallo térmico se encendía como debía. Además, se implementó una pequeña histéresis para evitar que el sistema cambiase rápidamente entre estado válido y fallo cuando la temperatura se encontraba cerca del límite.

9.4. Estados SAFE, ARMED y FIRE

El sistema parte inicialmente del estado SAFE. En este estado, las salidas hacia los optoacopladores permanecen desactivadas y la carga simulada no puede encenderse.

Cuando el operador activa ARM y la temperatura es correcta, el sistema pasa al estado ARMED. En este estado se activan las salidas ARM hacia los optoacopladores y se enciende el LED correspondiente.

Si durante la ventana temporal permitida se activa FIRE, el sistema pasa al estado FIRE. En ese momento se activan las salidas FIRE y la lógica digital permite que el MOSFET active el motor DC. Este motor representa la carga de terminación del sistema.

9.5. Limitaciones de la simulación

Aunque el sistema funciona correctamente a nivel conceptual, la simulación presenta varias limitaciones. La principal es que no se modela el comportamiento térmico real de la compuerta. Es decir, la apertura del servomotor no provoca una reducción automática de la temperatura simulada. La temperatura se introduce manualmente mediante la PT100, por lo que el sistema representa una relación directa entre temperatura medida y apertura, pero no un modelo térmico completo en bucle cerrado.

Además, las temporizaciones implementadas son aproximadas. El programa debe atender simultáneamente la lectura de temperatura, la visualización multiplexada, el control del servo y la lógica ARM/FIRE. Por tanto, los tiempos usados en la simulación son suficientes para representar el funcionamiento del sistema, pero no deben interpretarse como temporizaciones certificadas de un FTS real.

Otra limitación observada durante el desarrollo está relacionada con el uso de interruptores mantenidos para las señales ARM y FIRE. En un sistema real, estas señales estarían gestionadas mediante una lógica de mando más robusta, con enclavamientos, confirmaciones temporales y detección clara de flancos. En la simulación realizada, la secuencia ARM/FIRE se ha representado de forma simplificada, por lo que el objetivo principal es

mostrar la necesidad de una activación secuencial, no reproducir un sistema certificado de enclavamiento.

Finalmente, la carga explosiva real se ha sustituido por un motor DC, lo que permite visualizar la activación del sistema de forma segura dentro de Proteus.

10. Conclusiones

La realización de este proyecto nos ha permitido entender mejor cómo se integran distintos bloques electrónicos dentro de un mismo sistema de control. Aunque el sistema FTS implementado es una versión simplificada de uno real, el trabajo nos ha servido para relacionar conceptos vistos en la asignatura, como entradas analógicas, acondicionamiento de señal, temporizadores, registros, optoacopladores, lógica digital y control de actuadores.

Una de las partes más útiles ha sido comprobar que el diseño de un circuito no consiste solo en conectar componentes, sino en hacer que hardware y código funcionen de forma conjunta. Durante el desarrollo aparecieron problemas con el servomotor, el multiplexado de displays y la selección de los dígitos, lo que nos obligó a analizar señales, revisar conexiones y entender mejor el funcionamiento real de cada bloque.

También hemos aprendido la importancia de simplificar correctamente un sistema físico para poder simularlo. En nuestro caso, la compuerta de refrigeración responde a la temperatura, pero no hemos modelado el efecto real de enfriamiento que tendría sobre el sistema. Esto nos ha hecho ver la diferencia entre una simulación funcional en Proteus y un sistema real en bucle cerrado.

En conclusión, el proyecto nos ha ayudado a aplicar de forma práctica los conocimientos de la asignatura y a comprender mejor la relación entre sensores, microcontrolador, visualización y actuadores. Además, nos ha mostrado la importancia de depurar paso a paso, comprobar cada bloque por separado y justificar las simplificaciones realizadas.

11. Bibliografía

Referencias

- [1] Microchip Technology. (2020). *ATmega328P 8-bit AVR microcontroller data sheet*. Microchip Technology. <https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega328P-Data-Sheet-DS40002061B.pdf>
- [2] Texas Instruments. (2024). *TL082 JFET-input operational amplifier data sheet*. Texas Instruments. <https://www.ti.com/lit/ds/symlink/tl082.pdf>
- [3] Texas Instruments. (2024). *ULN2003A Darlington transistor array data sheet*. Texas Instruments. <https://www.ti.com/lit/ds/symlink/uln2003a.pdf>
- [4] Vishay. (2024). *PC817 optocoupler data sheet*. Vishay Intertechnology. <https://www.vishay.com/docs/83522/pc817.pdf>
- [5] Vishay. (2024). *1N4001-1N4007 general purpose rectifier data sheet*. Vishay Intertechnology. <https://www.vishay.com/docs/88503/1n4001.pdf>
- [6] Labcenter Electronics. (s. f.). *Proteus Design Suite*. Labcenter Electronics. <https://www.labcenter.com/>

- [7] National Aeronautics and Space Administration. (2010). *Range safety user requirements manual*. NASA. <https://www.nasa.gov/>
- [8] National Aeronautics and Space Administration. (s. f.). *Flight termination system and range safety*. NASA. <https://www.nasa.gov/>

A. Código completo del microcontrolador

En este anexo se incluye el código fuente del archivo `main.ino` del proyecto de Proteus. Esta versión incorpora la histéresis térmica, el bloqueo de la secuencia y la indicación de fallo mediante `tempready`, descritos en el cuerpo del trabajo.

Listado 1: Código fuente del ATmega328P incluido en el proyecto de Proteus.

```
1  #ifndef F_CPU
2  #define F_CPU 16000000UL
3  #endif
4
5  #include <avr/io.h>
6  #include <avr/interrupt.h>
7  #include <util/delay_basic.h>
8  #include <stdlib.h>
9  #include <math.h>
10
11 /* =====
12 1. VARIABLES GENERALES
13 ===== */
14
15 unsigned int VTemp;
16 float Vout, Vref, Vpt, RxT, Treal = 0.0;
17 int T = 0;
18 int apertura = 0; // apertura mostrada en display, 0-99 %
19 int servoCmd = 0; // mando del servo, 0-100 %
20
21 uint8_t armready = 0;
22 uint8_t fireready = 0;
23 uint8_t tempready = 0;
24 uint8_t bloqueo_secuencia = 0; // Enclavamiento para interruptores fijos
25
26 volatile uint8_t actualizar200ms = 0;
27
28 /* =====
29 2. CONSTANTES DEL PT100 Y UMBRALES
30 ===== */
31
32 float R17 = 100.0; // Ohm, rama superior referencia
33 float R18 = 100.0; // Ohm, rama superior PT100
34 float R19 = 72.6419; // Ohm, rama inferior referencia
35 float R20 = 56.0; // kOhm, ganancia diferencial
36 float R21 = 10.0; // kOhm, ganancia diferencial
37 float R0 = 100.0; // Ohm, PT100 a 0 °C
38 float alpha = 0.0039083; // coeficiente PT100
39
40 #define PT100_SIGN 1.0 // cambiar a -1.0 si la temperatura va invertida
41
42 #define TEMP_INICIO_REFRIGERACION 55.0 // desde aquí empieza a abrir la compuerta
43 #define TEMP_LIMITE_SERVO 75.0 // a esta temperatura servo al máximo
44 #define TEMP_LIMITE_SEGURIDAD 75.0 // a esta temperatura se bloquea FIRE
45 #define TEMP_MINIMA_SEGURIDAD -40.0 // límite inferior operativo
46
47 /* =====
48 3. MAPA DE PINES
49 ===== */
50
51 /* PORTC */
52 #define LED_ARMED PC1 // LED estado ARMED
53 #define LED_FIRE PC2 // LED estado FIRE
54 #define LED_SAFE PC3 // LED estado SAFE
55 #define DIG1 PC4 // selección dígito 1 del LEDMPX
56 #define CLK_4094 PC5 // reloj común de los 4094
57
58 /* PORTD */
```

```

59 #define ARM_IN          PD0          // entrada ARM, activa en LOW
60 #define FIRE_IN         PD1          // entrada FIRE, activa en LOW
61 #define LED_FALLO_TERM  PD2          // LED fallo térmico
62 #define DIG2             PD3          // selección dígito 2 del LEDMPX
63 #define OPTO_ARM1        PD4          // opto ARM 1
64 #define OPTO_ARM2        PD5          // opto ARM 2
65 #define OPTO_FIRE1       PD6          // opto FIRE 1
66 #define OPTO_FIRE2       PD7          // opto FIRE 2
67
68 /* PORTB */
69 #define DATA_TEMP       PB0          // dato 4094 temperatura
70 #define SERVO_PWM        PB1          // salida servo 0C1A
71 #define STB_4094         PB2          // strobe común 4094
72 #define SIGN_TEMP        PB3          // signo negativo temperatura
73 #define DATA_APERTURA  PB5          // dato 4094 apertura
74
75 /* =====
76 4. TABLA 7 SEGMENTOS, CÁTODO COMÚN
77 ===== */
78
79 unsigned char tabla7seg[10] = {
80     0b00111111,          // 0
81     0b00000110,          // 1
82     0b01011011,          // 2
83     0b01001111,          // 3
84     0b01100110,          // 4
85     0b01101101,          // 5
86     0b01111101,          // 6
87     0b00000111,          // 7
88     0b01111111,          // 8
89     0b01101111,          // 9
90 };
91
92 /* =====
93 5. PROTOTIPOS
94 ===== */
95
96 void espera_us(unsigned int us);
97 void espera_ms(unsigned int ms);
98
99 void actualizar_sistema(void);
100 void leer_temperatura(void);
101 void actualizar_tempready(void);
102 void calcular_apertura_y_servo(void);
103 void actualizar_led_fallo_termico(void);
104
105 void activar_arm(void);
106 void desactivar_arm(void);
107 void activar_fire(void);
108 void desactivar_fire(void);
109
110 void estado_safe(void);
111 void estado_armed(void);
112 void estado_fire(void);
113
114 void enviar_4094_doble(unsigned char datoTemp, unsigned char datoApertura);
115 void display_apertura(int dato);
116 void apagar_digitos(void);
117 void encender_dig1(void);
118 void encender_dig2(void);
119
120 void pulso_servo(int mando);
121 void servicio_visualizacion(void);
122
123 /* =====
124 6. SETUP

```

```

125  ===== */
126
127 void setup() {
128     DDRC = (1 << LED_ARMED) | (1 << LED_FIRE) | (1 << LED_SAFE) |
129           (1 << DIG1) | (1 << CLK_4094);    // PC0 ADC; PC1-PC5 salidas usadas
130     PORTC = 0x00;                          // salidas inicialmente a LOW
131
132     DDRD = (1 << LED_FALLO_TERM) | (1 << DIG2) |
133           (1 << OPTO_ARM1) | (1 << OPTO_ARM2) |
134           (1 << OPTO_FIRE1) | (1 << OPTO_FIRE2); // PD0/PD1 entradas; resto salidas
135     PORTD = (1 << ARM_IN) | (1 << FIRE_IN);    // pull-up ARM/FIRE, activos en LOW
136
137     DDRB = (1 << DATA_TEMP) | (1 << SERVO_PWM) |
138           (1 << STB_4094) | (1 << SIGN_TEMP) |
139           (1 << DATA_APERTURA);              // PB0,PB1,PB2,PB3,PB5 salidas
140     PORTB = 0x00;                          // salidas inicialmente a LOW
141
142     ADCSRA = 0b10000111;                  // ADC ON, prescaler 128
143     ADMUX = 0b01000000;                   // AVCC como referencia, ADC0
144
145     TCCR1A = (1 << COM1A1);                // Timer1 PWM no inversor en OC1A
146     TCCR1B = (1 << WGM13) | (1 << CS11);    // modo 8, prescaler 8
147     ICR1 = 20000;                         // periodo 20 ms aprox. para servo
148
149     TCCR2A = 0x00;                        // Timer2 modo normal
150     TCCR2B = (1 << CS22) | (1 << CS21) | (1 << CS20); // prescaler 1024
151     TIMSK2 |= (1 << TOIE2);               // interrupción overflow Timer2
152
153     desactivar_arm();                     // optos ARM apagados
154     desactivar_fire();                   // optos FIRE apagados
155     estado_safe();                       // estado inicial SAFE
156     apagar_digitos();                   // multiplexado apagado al inicio
157
158     actualizar_sistema();                 // primera lectura del sistema
159     pulso_servo(0);                      // servo cerrado al inicio
160
161     sei();                               // habilita interrupciones globales
162 }
163
164 /* =====
165 7. LOOP PRINCIPAL (BLINDADO PARA INTERRUPTORES Y TEMPERATURA)
166 ===== */
167
168 void loop() {
169     unsigned int i, j;
170
171     if (actualizar200ms) {                // actualización periódica
172         actualizar200ms = 0;
173         actualizar_sistema();
174     }
175
176     servicio_visualizacion();             // refresco continuo del display
177
178     // --- 1. LÓGICA DE RESETEO DE SEGURIDAD ---
179     // Si la palanca ARM se baja físicamente (OFF), limpiamos el bloqueo
180     if ((PIND & (1 << ARM_IN)) != 0) {
181         bloqueo_secuencia = 0;
182     }
183
184     // --- 2. SECUENCIA DE LANZAMIENTO ---
185     // Solo inicia si ARM se pone en ON y NO estamos bloqueados
186     if (((PIND & (1 << ARM_IN)) == 0) && (bloqueo_secuencia == 0)) {
187         actualizar_sistema();
188
189         // REGLA ANTI-FUEGO PREMATURO: Si FIRE ya está subido, bloqueamos.
190         if ((PIND & (1 << FIRE_IN)) == 0) {

```

```

191     bloqueo_secuencia = 1;
192     return; // Aborta esta vuelta del loop
193 }
194
195 if (tempready == 1) {
196     armready = 1;
197     fireready = 0;
198
199     activar_arm();
200     estado_armed();
201
202     for (i = 0; i < 10000; i++) { // Ventana de 10 s aprox.
203         actualizar_sistema();
204         servicio_visualizacion();
205
206         // ABORTO INSTANTÁNEO: El operador baja la palanca ARM
207         if ((PIND & (1 << ARM_IN)) != 0) {
208             break;
209         }
210
211         // ABORTO TÉRMICO
212         if (tempready == 0) {
213             break;
214         }
215
216         // INTENTO DE DISPARO
217         if ((tempready == 1) &&
218             (armready == 1) &&
219             ((PIND & (1 << FIRE_IN)) == 0) &&
220             (fireready == 0)) {
221             fireready = 1;
222         }
223
224         if (fireready == 1) {
225             activar_fire();
226             estado_fire();
227
228             for (j = 0; j < 500; j++) { // 500ms de fuego aprox.
229                 actualizar_sistema();
230                 servicio_visualizacion();
231                 espera_ms(1);
232             }
233
234             desactivar_fire();
235             fireready = 0;
236
237             // Fuego completado: Forzamos bloqueo hasta que se baje ARM
238             bloqueo_secuencia = 1;
239             break;
240         }
241
242         espera_ms(1);
243     }
244
245     // Si salimos del bucle (por tiempo, disparo, o aborto), bloqueamos y limpiamos
246     bloqueo_secuencia = 1;
247     armready = 0;
248     fireready = 0;
249
250     desactivar_arm();
251     desactivar_fire();
252     estado_safe();
253 }
254 }
255 }
256

```

```

257 /* =====
258 8. TIMER2: FLAG DE ACTUALIZACIÓN CADA ~200 ms
259 ===== */
260
261 ISR(TIMER2_OVF_vect) {
262     static uint8_t contador200ms = 0;
263
264     contador200ms++; // cuenta overflows de Timer2
265
266     if (contador200ms >= 12) { // 12 overflows ~ 200 ms
267         actualizar200ms = 1; // pide actualización en loop
268         contador200ms = 0;
269     }
270 }
271
272 /* =====
273 9. RETARDOS
274 ===== */
275
276 void espera_us(unsigned int us) {
277     while (us > 0) {
278         _delay_loop_2(4); // aprox. 1 us a 16 MHz
279         us--;
280     }
281 }
282
283 void espera_ms(unsigned int ms) {
284     while (ms > 0) {
285         _delay_loop_2(4000); // aprox. 1 ms a 16 MHz
286         ms--;
287     }
288 }
289
290 /* =====
291 10. ACTUALIZACIÓN GENERAL DEL SISTEMA
292 ===== */
293
294 void actualizar_sistema(void) {
295     leer_temperatura(); // calcula Treal y T
296     actualizar_tempready(); // comprueba rango operativo
297     calcular_apertura_y_servo(); // calcula apertura y servo
298     actualizar_led_fallo_termico(); // LED fallo térmico
299 }
300
301 /* =====
302 11. LECTURA Y CÁLCULO DE TEMPERATURA
303 ===== */
304
305 void leer_temperatura(void) {
306     float G;
307
308     ADMUX = 0b01000000; // selecciona ADC0
309     ADSCRA |= (1 << ADSC); // inicia conversión ADC
310     while ((ADCSRA & (1 << ADSC)) != 0); // espera fin de conversión
311
312     VTemp = ADC; // lectura ADC 0-1023
313     Vout = 5.0 * VTemp / 1023.0; // tensión del acondicionador
314
315     G = R20 / R21; // ganancia del amplificador
316     Vref = 5.0 * R19 / (R17 + R19); // tensión de referencia del puente
317     Vpt = Vref + PT100_SIGN * (Vout / G); // tensión equivalente del PT100
318
319     if (Vpt < 0.01) Vpt = 0.01; // evita división por cero
320     if (Vpt > 4.99) Vpt = 4.99; // limita al rango físico
321
322     RxT = R18 * Vpt / (5.0 - Vpt); // resistencia equivalente PT100

```

```

323     Treal = (RxT - RO) / (RO * alpha);           // temperatura calculada
324     T      = (int)(Treal);                       // temperatura entera para display
325 }
326
327 /* =====
328 12. CONTROL TÉRMICO (CON HISTÉRESIS)
329 ===== */
330
331 void actualizar_tempready(void) {
332     if (tempready == 1) {
333         // Si el sistema está operativo, comprobamos si sobrepasa los límites estrictos
334         if ((Treal < TEMP_MINIMA_SEGURIDAD) || (Treal >= TEMP_LIMITE_SEGURIDAD)) {
335             tempready = 0;
336             fireready = 0;
337             desactivar_fire();
338         }
339     } else {
340         // Histéresis: Si estaba en fallo térmico, necesita un margen de 2 grados para resetearse
341         if ((Treal >= TEMP_MINIMA_SEGURIDAD + 2.0) && (Treal < TEMP_LIMITE_SEGURIDAD - 2.0)) {
342             tempready = 1;
343         }
344     }
345 }
346
347 void calcular_apertura_y_servo(void) {
348     if (Treal <= TEMP_INICIO_REFRIGERACION) {
349         apertura = 0;                               // compuerta cerrada
350         servoCmd = 0;                               // servo mínimo
351     }
352     else if (Treal < TEMP_LIMITE_SERVO) {
353         apertura = (int)((Treal - TEMP_INICIO_REFRIGERACION) * 99.0 /
354             (TEMP_LIMITE_SERVO - TEMP_INICIO_REFRIGERACION));
355
356         servoCmd = (int)((Treal - TEMP_INICIO_REFRIGERACION) * 100.0 /
357             (TEMP_LIMITE_SERVO - TEMP_INICIO_REFRIGERACION));
358     }
359     else {
360         apertura = 99;                               // apertura máxima mostrada
361         servoCmd = 100;                              // servo máximo
362     }
363
364     if (apertura < 0) apertura = 0;                 // saturación inferior display
365     if (apertura > 99) apertura = 99;              // saturación superior display
366
367     if (servoCmd < 0) servoCmd = 0;                // saturación inferior servo
368     if (servoCmd > 100) servoCmd = 100;            // saturación superior servo
369
370     pulso_servo(servoCmd);                         // actualiza PWM servo
371 }
372
373 void actualizar_led_fallo_termico(void) {
374     // Si la temperatura no está lista, encendemos la luz de fallo. Sincronización perfecta.
375     if (tempready == 0) {
376         PORTD |= (1 << LED_FALLO_TERM);           // LED fallo térmico ON
377     } else {
378         PORTD &= ~(1 << LED_FALLO_TERM);           // LED fallo térmico OFF
379     }
380 }
381
382 /* =====
383 13. ARM / FIRE
384 ===== */
385
386 void activar_arm(void) {
387     PORTD |= (1 << OPTO_ARM1) | (1 << OPTO_ARM2); // activa optos ARM
388 }

```

```

389
390 void desactivar_arm(void) {
391     PORTD &= ~((1 << OPTO_ARM1) | (1 << OPTO_ARM2)); // apaga optos ARM
392 }
393
394 void activar_fire(void) {
395     PORTD |= (1 << OPTO_FIRE1) | (1 << OPTO_FIRE2); // activa optos FIRE
396 }
397
398 void desactivar_fire(void) {
399     PORTD &= ~((1 << OPTO_FIRE1) | (1 << OPTO_FIRE2)); // apaga optos FIRE
400 }
401
402 /* =====
403 14. Leds de estado
404 ===== */
405
406 void estado_safe(void) {
407     PORTC &= ~(1 << LED_ARMED); // ARMED OFF
408     PORTC &= ~(1 << LED_FIRE); // FIRE OFF
409     PORTC |= (1 << LED_SAFE); // SAFE ON
410 }
411
412 void estado_armed(void) {
413     PORTC |= (1 << LED_ARMED); // ARMED ON
414     PORTC &= ~(1 << LED_FIRE); // FIRE OFF
415     PORTC &= ~(1 << LED_SAFE); // SAFE OFF
416 }
417
418 void estado_fire(void) {
419     PORTC |= (1 << LED_ARMED); // ARMED ON
420     PORTC |= (1 << LED_FIRE); // FIRE ON
421     PORTC &= ~(1 << LED_SAFE); // SAFE OFF
422 }
423
424 /* =====
425 15. Envío a 4094
426 ===== */
427
428 void enviar_4094_doble(unsigned char datoTemp, unsigned char datoApertura) {
429     unsigned char i;
430
431     for (i = 0; i < 8; i++) {
432         if (datoTemp & 0x01) PORTB |= (1 << DATA_TEMP); // bit temperatura
433         else PORTB &= ~(1 << DATA_TEMP);
434
435         if (datoApertura & 0x80) PORTB |= (1 << DATA_APERTURA); // bit apertura
436         else PORTB &= ~(1 << DATA_APERTURA);
437
438         espera_us(2);
439
440         PORTC |= (1 << CLK_4094); // flanco subida CLK
441         espera_us(2);
442         PORTC &= ~(1 << CLK_4094); // flanco bajada CLK
443         espera_us(2);
444
445         datoTemp = datoTemp / 2; // temperatura LSB primero
446         datoApertura = datoApertura * 2; // apertura MSB primero
447     }
448
449     PORTB |= (1 << STB_4094); // STROBE activa salida
450     espera_us(2);
451     PORTB &= ~(1 << STB_4094);
452     espera_us(2);
453 }
454

```

```

455 /* =====
456 16. DISPLAY MULTIPLEXADO DE APERTURA
457 ===== */
458
459 void apagar_digitos(void) {
460     PORTC &= ~(1 << DIG1);           // DIG1 OFF
461     PORTD &= ~(1 << DIG2);           // DIG2 OFF
462 }
463
464 void encender_dig1(void) {
465     PORTC |= (1 << DIG1);           // DIG1 ON
466 }
467
468 void encender_dig2(void) {
469     PORTD |= (1 << DIG2);           // DIG2 ON
470 }
471
472 void display_apertura(int dato) {
473     unsigned char dec, uni, datoTempBCD;
474     int tempAbs;
475
476     if (dato < 0) dato = 0;           // evita negativos
477     if (dato > 99) dato = 99;         // limita a 2 dígitos
478
479     dec = dato / 10;                  // decenas
480     uni = dato % 10;                 // unidades
481
482     if (T < 0) {
483         PORTB |= (1 << SIGN_TEMP);   // signo negativo ON
484         tempAbs = abs(T);
485     } else {
486         PORTB &= ~(1 << SIGN_TEMP);   // signo negativo OFF
487         tempAbs = T;
488     }
489
490     if (tempAbs > 99) tempAbs = 99;   // temperatura limitada a 2 dígitos
491
492     datoTempBCD = ((tempAbs / 10) << 4) + (tempAbs % 10); // temperatura en BCD
493
494     apagar_digitos();                // apaga dígitos antes de cambiar segmentos
495     espera_us(200);
496
497     enviar_4094_doble(datoTempBCD, tabla7seg[dec]); // carga decenas apertura
498     encender_dig1();                  // enciende primer dígito
499     espera_ms(4);
500
501     apagar_digitos();                // apaga antes de cambiar al otro
502     espera_us(200);
503
504     enviar_4094_doble(datoTempBCD, tabla7seg[uni]); // carga unidades apertura
505     encender_dig2();                  // enciende segundo dígito
506     espera_ms(4);
507
508     apagar_digitos();                // evita ghosting
509 }
510
511 /* =====
512 17. SERVO
513 ===== */
514
515 void pulso_servo(int mando) {
516     unsigned int us;
517
518     if (mando < 0) mando = 0;         // mínimo servo
519     if (mando > 100) mando = 100;     // máximo servo
520

```

```

521     us = 1000 + ((unsigned long)mando * 556) / 100; // pulso calibrado del servo
522
523     cli();                                     // evita escritura interrumpida de OCR1A
524     OCR1A = us;                               // actualiza posición servo
525     sei();                                     // reactiva interrupciones
526 }
527
528 /* =====
529 18. SERVICIO DE VISUALIZACIÓN
530 ===== */
531
532 void servicio_visualizacion(void) {
533     display_apertura(apertura);               // refresca display de apertura
534 }

```